
Introducción al Aprendizaje Automático

1er cuatrimestre 2025



Universidad
Nacional
de San Martín

Escalado de datos

Escalado de Datos

- Es una buena práctica, antes de entrenar cualquier modelo, escalar los datos. En general, se utiliza el Z-Score para la mayoría de los atributos.
- El escalado es particularmente importante cuando se trate de modelos que involucren distancias u operaciones matemáticas donde los órdenes de magnitud pueden confundir al modelo (kNN, regresión lineal y logística). No lo es tanto en modelos como árboles de decisión.
- Si hay un atributo binario numérico (0/1), no hace falta escalarlo.
- ¡Cuidado cuando hay atributos que comparten unidades! Por ejemplo, dos atributos medidos en metros.

Escalado

1. Escalado Mínimo-Máximo
2. Estandarización de datos o escalado por Z-Score
3. Escalados no-lineales

Escalado Mínimo-Máximo

- 1. Escalado Mínimo-Máximo:** en general se usa cuando sabemos que nuestra variable puede tomar valores entre un mínimo y un máximo bien definidos.

Ejemplo: las notas de un examen de 0 a 10. En esos casos, en general se suele hacer un escalado lineal de forma tal de hacer la siguiente asignación:

Valor máximo $\rightarrow 1$

Valor mínimo $\rightarrow 0$

y al resto de los valores se les asigna un valor intermedio. Para el caso del examen con notas del 0 al 10, podría quedar así:

10 $\rightarrow 1$

9 $\rightarrow 0.9$

...

1 $\rightarrow 0.1$

0 $\rightarrow 0$

En Scikit-Learn, existe una clase [MinMaxScaler\(\)](#) del módulo *preprocessing* que lo implementa.

Z-Score

Tenemos un conjunto de números $x_1, x_2, x_3, \dots, x_n$. Su media es μ , y su desviación estándar σ .

$$Z = (x_i - \mu) / \sigma$$

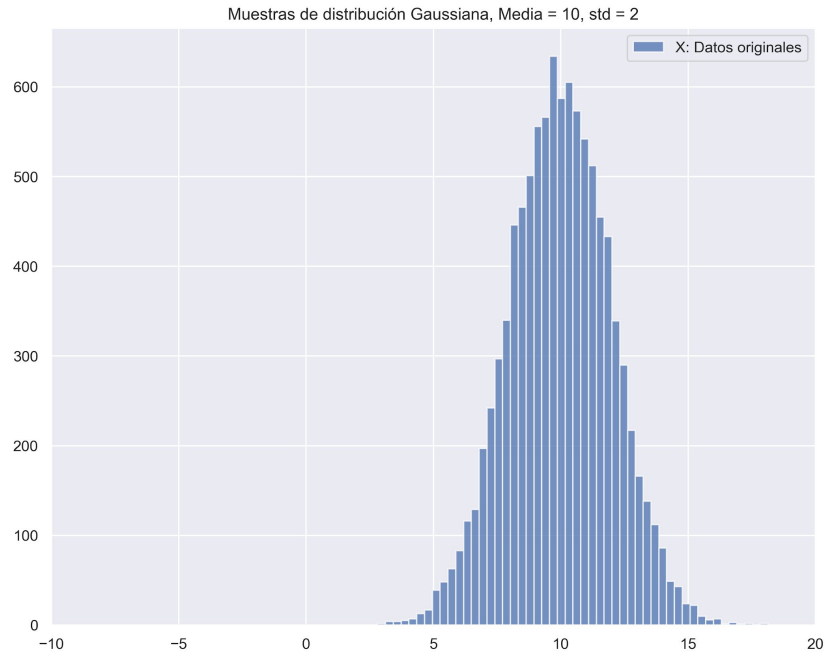
Es una medida de cuánto se desvía un valor del promedio, medido en desviaciones estándar.

Ejemplo: $x_1 = 1, x_2 = 2, x_3 = 1.5$

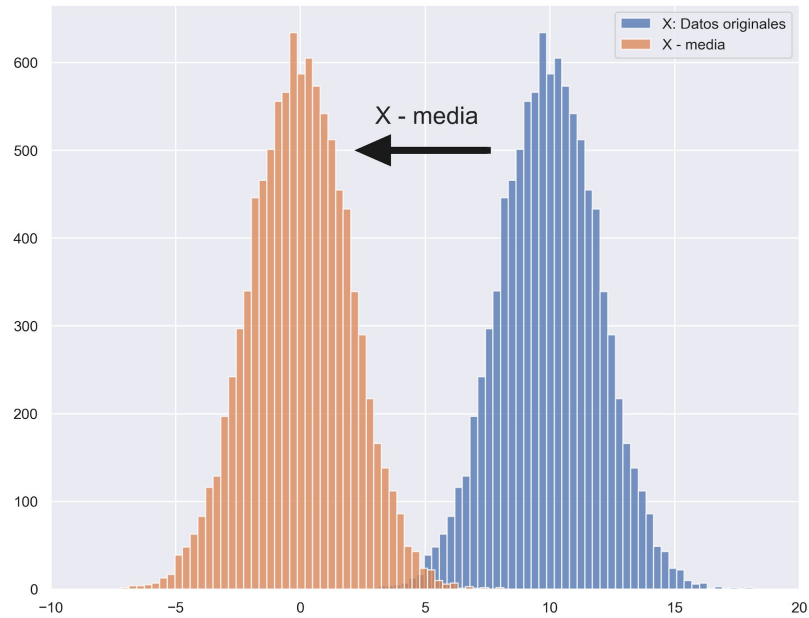
- Media, $\mu = 1.5$
- Desviación estándar, $\sigma = 0.5$

$x_1 = 1$	$\longrightarrow$	$z_1 = (1 - 1.5) / 0.5 = -1$
$x_2 = 2$	$\longrightarrow$	$z_2 = (2 - 1.5) / 0.5 = 1$
$x_3 = 1.5$	$\longrightarrow$	$z_3 = (1.5 - 1.5) / 0.5 = 0$

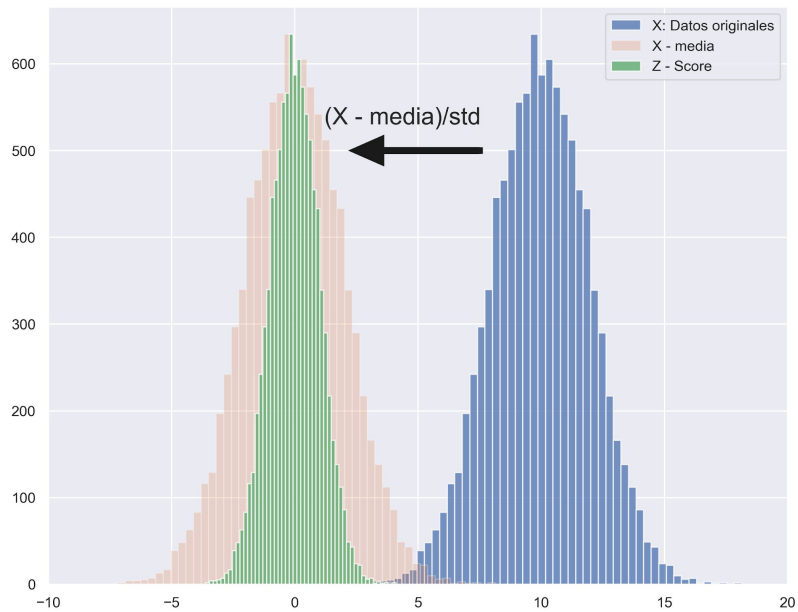
Z-Score



Z-Score



Z-Score



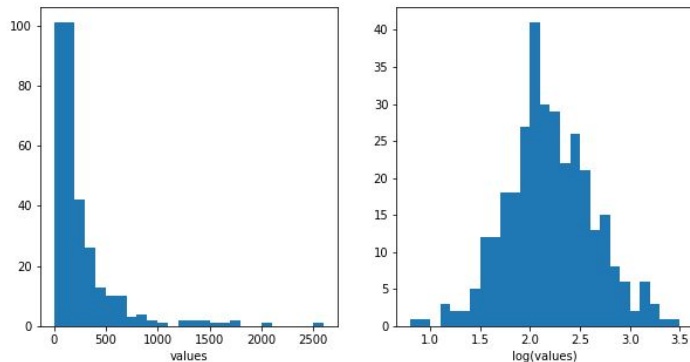
Z-Score

- El Z-Score es una medida de cuán lejos está un dato del promedio de la muestra a cual pertenece, medido en desviaciones estándar.
- Es una medida muy usada para hablar de variables y distribuciones, en particular en ámbitos como Medicina.
- Nos sirve para **Escalado de Datos**. A veces lo podrán encontrar por el nombre de Estandarización o Normalización.
- También sirve para aplicar la regla de las tres sigmas, ya que luego de normalizar los datos, es simplemente usar una máscara $|Z| > 3$ para seleccionar valores atípicos.
- En Scikit-Learn, existe una clase [StandardScaler\(\)](#) del módulo *preprocessing* que lo implementa.

Escalados no-lineales

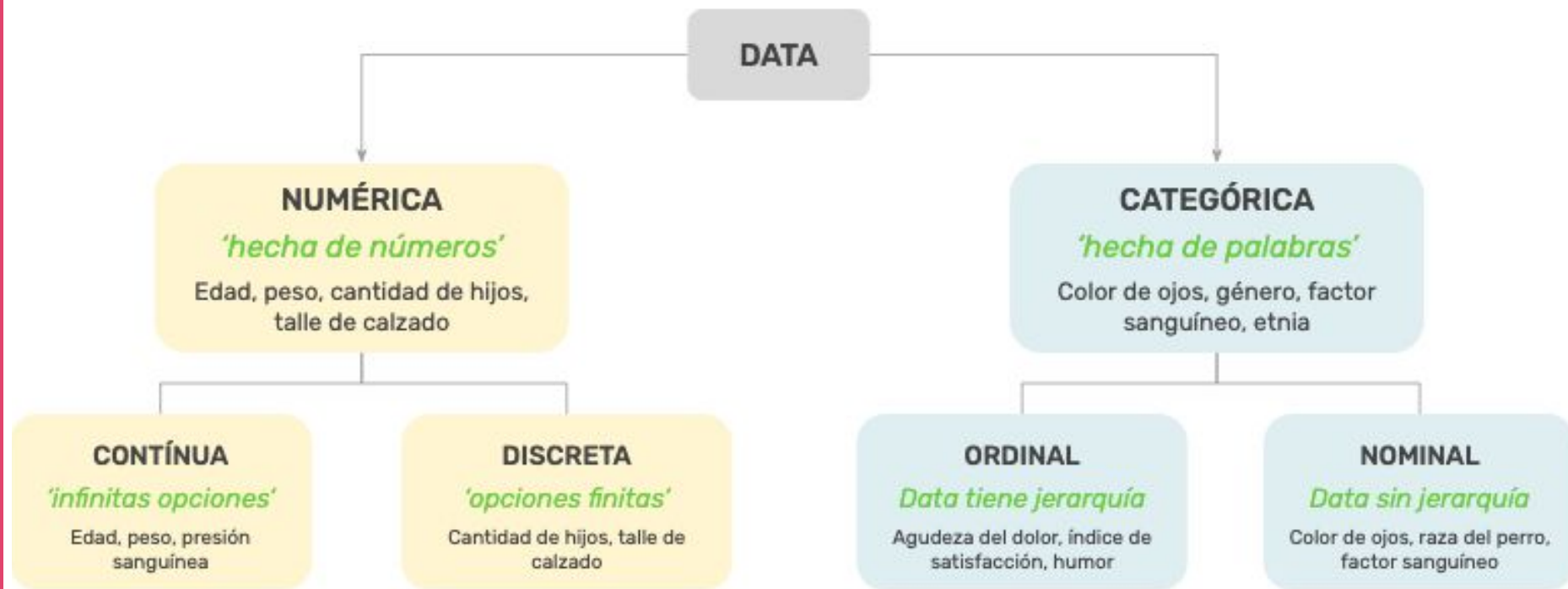
Muchos modelos andan *mejor* cuando los atributos siguen distribuciones normales. Pero éste muchas veces no es el caso. A veces las distribuciones no son simétricas, sino que tienen una *cola*, o, peor aún, siguen una ley de potencias. En esos casos, una transformación lineal como el Z-Score no servirá de mucho. Hay dos transformaciones que suelen ser útiles:

1. En el caso de una distribución que siga una ley de potencias, tomar el logaritmo de los valores transforma la distribución en algo muy parecido a una normal.
2. Si tomar logaritmo es muy drástico, ya sea porque la distribución tiene una cola pero no llega a ser una ley de potencias, tomar la raíz cuadrada suele ser un método efectivo.



Tipos de datos

Tipos de datos



Variables numéricas

Son aquellas variables que se miden o se cuentan (discretas o continuas).

- Hay una relación de orden entre ellas
- Se pueden sumar (en algunas circunstancias)

Ejemplos:

- Edad, Altura y Peso de una persona
- Puntaje, precio de un vino
- Valor de un pasaje
- Etc.

Tratamiento

En general, ya vienen en un formato “cómodo” para trabajar, pero a veces queremos agruparlas según grupos o rangos.

Ejemplo: agrupar edades en rangos (bebés, niños, adolescentes, adultos, ancianos)

→ **Discretización y Binning**

Variables ordinales

Sus posibles valores son categorías, pero hay una relación de orden.

¡Notar que no se pueden sumar!

Ejemplos:

- Tamaño de una prenda de ropa: XS, S, M, L, XL
- Tipo de Nafta por octanaje: 95, 98, más de 98.
- Rangos etarios: bebé, niño/a, adolescente, adulto/a, anciano/a

Tratamiento

Podemos hacer una asignación a número enteros manteniendo el orden:

$S \rightarrow 0$

$M \rightarrow 1$

$L \rightarrow 2$

Pero, ¡cuidado!, recordar que no se pueden sumar.

Variables nominales

Sus posibles valores pertenecen a una de varias categorías.

- Las categorías no siguen una relación de orden
- Ninguna es mayor que otra

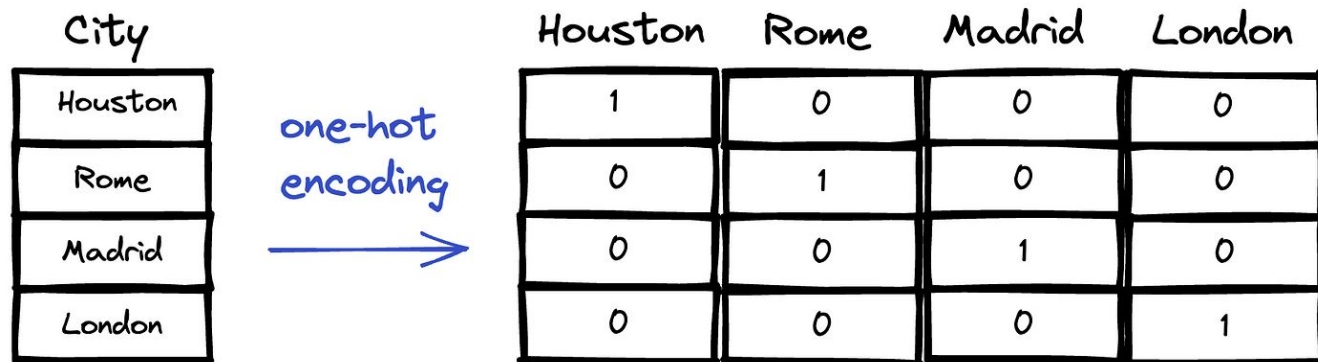
Ejemplos:

- Nacionalidad
- Tipo de vino
- Color de una prenda de ropa
- Género: femenino, masculino, no binario, etc.

Tratamiento

- Llevar a variables dummies/One-Hot Encoding.

Variables nominales



Ojo con:

- Espacio/Memoria RAM
- Tiempo de entrenamiento
- Problemas de dimensionalidad y Sobreajuste
- ¿Qué sucede con categorías nueva?

En Scikit-Learn, existe una clase [OneHotEncoder\(\)](#) del módulo *preprocessing* que lo implementa.

Resumiendo

Conversión de variables: Los modelos sólo entienden de números.
¿Si los atributos no son números?

Tipos de variables a tratar:

- Numéricas: edad, altura, puntaje.
- Ordinales: tamaño de una prenda de ropa.
- Categóricas/nominales: nacionalidad, color de una prenda de ropa.
- Y más...

Tratamiento

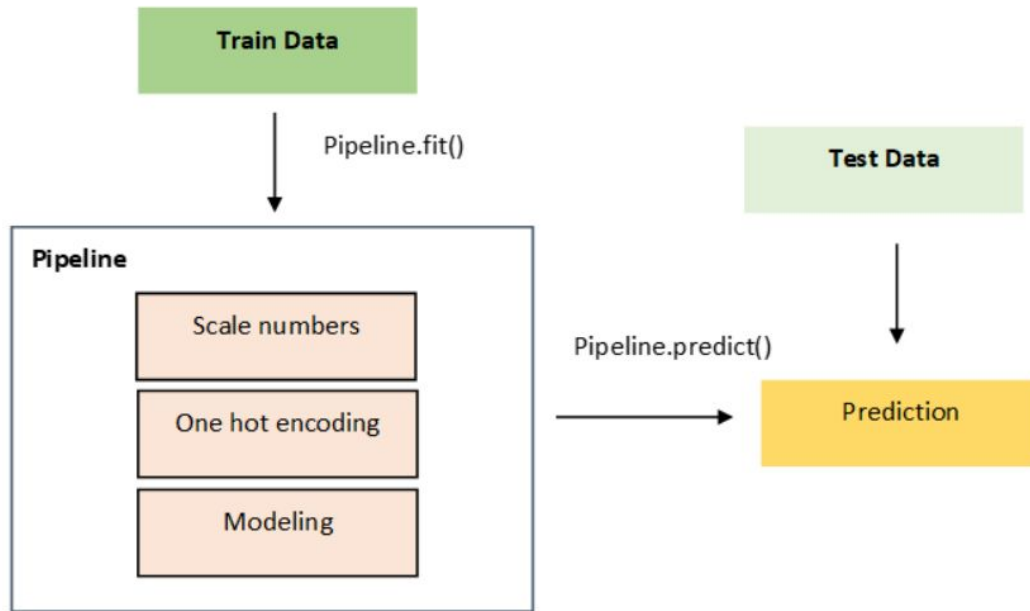
- Discretización/binning
- "Labelización"¹
- Variables dummies/One-Hot encoding
- Y más...

¹ Tal vez inventamos una palabra

Pipelining y Transformaciones por Columnas

Pipelining

El **pipelining** es una técnica que organiza el proceso de transformación de datos y entrenamiento del modelo en una secuencia estructurada de pasos. Cada paso en la tubería (pipeline) realiza una tarea específica (limpieza de datos, transformación de datos y modelado), en forma secuencial



Pipelining

La herramienta más común de SciKit-Learn para componer estimadores es [Pipeline\(\)](#).

- Todos los *steps* deben ser [transformers](#).
- El último paso (*step*) también puede ser un [predictor](#).
- Se pueden acceder los *steps* con los métodos [get_params\(\)](#) y [set_params\(\)](#)

```
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

pipe = Pipeline([('scaler', StandardScaler()), ('lr', LogisticRegression())])

# The pipeline can be used as any other estimator
# and avoids leaking the test set into the train set
pipe.fit(X_train, y_train).score(X_test, y_test)
```

Pipelining

Se pueden configurar los parámetros de los diferentes *steps* usando el nombre del *step* y el nombre del parámetro separado por doble underscore

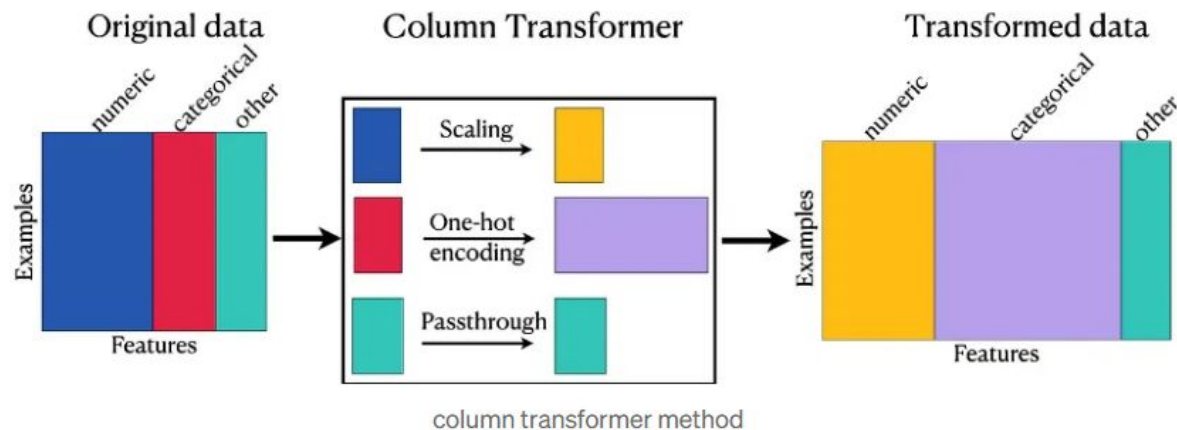
- `pipe.set_params(lr__ penalty=None)`

También se puede acceder a los atributos de los *steps*

- `pipe.named_steps['lr'].intercept_`

Transformaciones por columnas

La herramienta [ColumnTransformer](#) facilita la realización de diferentes transformaciones a distintas columnas de datos.



Transformaciones por columnas

La herramienta [ColumnTransformer](#) facilita la realización de diferentes transformaciones a distintas columnas de datos.

```
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

preproc = ColumnTransformer(
    [ ('scaler', StandardScaler(), pgs_numeric),
      ('one-hot', OneHotEncoder(), pgs_cat),
    ])

pipe = Pipeline([ ('preproc', preproc), ('lr', LogisticRegression()) ])

# The pipeline can be used as any other estimator
# and avoids leaking the test set into the train set
pipe.fit(X_train, y_train).score(X_test, y_test)
```